

--

记	
分	

实验项目名称: 实验二-音频单模态机器学习

实验项目名称：实验二-音频单模态机器学习

一.实验背景与目标

随着人工智能技术的不断发展，情感识别已经成为人机交互、智能语音助手、情感计算等领域的重要研究方向。情感识别旨在通过分析人类的语音、表情、文本等输入信号，识别并理解其情感状态。在本实验中，更加专注于音频单模态情感识别，目标是利用机器学习方法通过音频特征的自动提取与分析，建立一个能够准确识别多种情感状态的模型。具体来说，本实验希望通过音频信号中的语音特征，准确识别四种情感状态：愤怒、快乐/兴奋、中性和悲伤。

实验目标包括：

- 通过音频信号的特征提取，获取情感相关特征信息；
- 使用机器学习算法训练情感分类模型；
- 评估所构建模型的准确性与鲁棒性，并且得到最终的test_result文件。

二.实验方法

本实验采用多种机器学习方法，结合音频信号处理技术，通过提取语音信号中的时域和频域特征进行情感分类，通过提取音频信号的特征，训练模型以识别四种情感状态：愤怒、快乐/兴奋、中性、悲伤，而提高最终模型性能的关键在于数据预处理、特征提取和最终模型训练。

具体的实验流程包括以下几个主要步骤：

- 数据预处理**：对原始音频数据进行降噪、分帧等预处理操作，以便提取高质量的特征。
- 特征提取**：使用 opensmile-python 库提取音频的多种特征，包括 梅尔频率倒谱系数（MFCC）、零交叉率（ZCR）、谱质心等，这些特征能够有效反映情感的声学特征。
- 模型训练**：基于提取的音频特征，采用传统的机器学习分类算法进行模型训练，利用训练集和验证集进行交叉验证，以优化超参数。
- 性能评估**：通过准确率、精确率、召回率、F1 分数等常用评价指标对模型在测试集上的性能进行评估。

下面进行详细介绍：

数据预处理

数据预处理是音频信号处理中至关重要的一步，它的主要目的是去除噪声并准备高质量的输入数据，以便后续的特征提取与模型训练。针对本实验的音频数据集，预处理过程包括以下几个关键步骤：

- **降噪**：音频信号中的噪声（如背景环境噪音、设备噪音等）会影响情感识别的准确性。为了减少噪音的干扰，采用了噪声抑制算法，例如基于频谱减法的噪声消除技术（Spectral Subtraction）或者 Wiener 滤波器，以提高音频信号的信噪比。
- **分帧**：音频信号通常是连续的，但为了提取时域和频域特征，我们需要将音频信号分为短时帧。每一帧的长度一般设置为20-40毫秒，并且相邻帧之间有一定的重叠（如50%的重叠），这种方式有助于捕捉信号的动态变化。
- **预加重**：预加重是一种频谱平滑方法，主要用于增强高频成分，补偿音频信号中高频部分的衰减。通常通过简单的高通滤波器来进行预加重处理。

在本实验中，仅尝试使用 `noisereduce` 进行降噪处理，在比较嘈杂的环境下，特别是 dev 数据集上，使得主要说话人的声音更加清晰，使其情感特征更容易进行分析，但是后续特征提取上区别不大，说明 `opensmile` 中的 eGeMAPS 特征集特征提取效果已经很好。

```
import noisereduce as nr
import librosa
import os
import soundfile as sf # 导入 soundfile 库

# 降噪函数
def denoise_audio(input_folder, output_folder):
    # 计数器
    processed_count = 0

    for filename in os.listdir(input_folder):
        # 检查是否是音频文件（这里只处理 .wav 格式的文件）
        if filename.endswith('.wav'):
            file_path = os.path.join(input_folder, filename)
            output_path = os.path.join(output_folder, filename)

            # 加载音频文件
            audio, sr = librosa.load(file_path, sr=None)

            # 使用 noisereduce 降噪
            reduced_noise_audio = nr.reduce_noise(y=audio, sr=sr)

            # 保存降噪后的音频
            sf.write(output_path, reduced_noise_audio, sr)

            # 更新计数器
            processed_count += 1

        # 每处理100个文件输出信息
        if processed_count % 100 == 0:
            print(f"已处理 {processed_count} 个文件: {input_folder}")

# 使用示例：对 train、dev 和 test 文件夹中的所有 WAV 文件进行降噪
train_folder = "./train/"
dev_folder = "./dev/"
test_folder = "./test/"
```

```
output_train_folder = "./denoised_train/"
output_dev_folder = "./denoised_dev/"
output_test_folder = "./denoised_test/"

# 对 train、dev 和 test 文件夹中的音频文件进行降噪
denoise_audio(train_folder, output_train_folder)
denoise_audio(dev_folder, output_dev_folder)
denoise_audio(test_folder, output_test_folder)
```

特征提取

使用opensmile库，eGeMAPS特征集进行特征提取，针对使用的eGeMAPS特征集进行详细研究

特征集特征分析

一：LLDs特征和HSFs特征

(1) 首先区分一下frame和utterance，**frame就是一帧语音，utterance是一段语音**，是比帧高一级的语音单位，通常指一句话，一个语音样本。**utterance由多帧语音组成**，通常对一个utterance做分帧来得到多帧信号。

(2) LLDs (low level descriptors) LLDs指的是手工设计的一些低水平特征，一般是在一帧语音上进行的计算，是用来表示一帧语音的特征。

(3) HSFs (high level statistics functions) 是在LLDs的基础上做一些统计而得到的特征，比如均值，最大值等等。HSFs是对utterance上的多帧语音做统计，所以是用来表示一个utterance的特征。

二：GeMAPS特征集

(1) GeMAPS特征集总共62个特征，这62个都是HSF特征，是由18个LLD特征计算得到。18个LLD特征包括6个频率相关特征，3个能量/振幅相关特征，9个谱特征。

- 基音F0的概念：通常记作F0（F0一般也指基音频率），一般的声音都是由发音体发出的一系列频率、振幅各不相同的振动复合而成的。这些振动中有一个频率最低的振动，由它发出的音就是基音，其余为泛音。
- 6个频率相关特征包括：Pitch（log F0，在半音频率尺度上计算，从27.5Hz开始）；Jitter（单个连续基音周期内的偏差，偏差衡量的是观测变量与特定值的差，如果没有指明特定值通常使用的是变量的均值）；前三个共振峰的中心频率，第一个共振峰的带宽。
- 3个能量/振幅的特征包括：Shimmer（相邻基音周期间振幅峰值之差），Loudness（从频谱中得到的声音强度的估计，可以根据能量来计算），HNR（Harmonics-to-noise）信噪比。
- 9个谱特征包括，Alpha Ratio（50-1000Hz的能量和除以1-5kHz的能量和），Hammarberg Index（0-2kHz的最强能量峰除以2-5kHz的最强能量峰），Spectral Slope 0-500 Hz and 500-1500 Hz（对线性功率谱的两个区域0-500 Hz和500-1500 Hz做线性回归得到的两个斜率），Formant 1, 2, and 3 relative energy（前三个共振峰的中心频率除以基音的谱峰能量），Harmonic difference H1-H2（第一个基音谐波H1的能量除以第二个基音谐波的的能量），Harmonic difference H1-A3（第一个基音谐波H1的能量除以第三个共振峰范围内的最高谐波能量）。

(4) 对18个LLD做统计，计算的时候是对3帧语音做symmetric moving average。

首先计算算术平均和coefficient of variation（计算标准差然后用算术平均规范化），得到36个统计特征。

然后对loudness和pitch运算8个函数，20百分位，50百分位，80百分位，20到80百分位之间的range，上升/下降语音信号的斜率的均值和标准差，得到16个统计特征。

上面的函数都是对voiced regions（非零的F0）做的。对Alpha Ratio, Hammarberg Index, Spectral Slope 0-500 Hz and 500-1500 Hz做算术平均得到4个统计特征。另外还有6个时间特征，每秒loudness 峰的个数，连续voiced regions（F0>0）的平均长度和标准差，unvoiced regions（F0=0）的平均长度和标准差，每秒voiced regions的个数。36+16+4+6得到62个特征。

三：eGeMAPS特征集

(1) eGeMAPS是GeMAPS的**扩展**，在18个LLDs的基础上加了一些特征，包括5个谱特征：MFCC1-4和Spectral flux（两个相邻帧的频谱差异）和2个频率相关特征：第二个共振峰和第三个共振峰的带宽。

(2) 对这扩展的7个LLDs做算术平均和coefficient of variation（计算标准差然后用算术平均规范化）可以得到14个统计特征。对于共振峰带宽只在voiced region做，对于5个谱特征在voiced region和unvoiced region一起做。

(3) 另外，只在unvoiced region计算spectral flux的算术平均，然后只在voiced region计算5个谱特征的算术平均和coefficient of variation，得到11个统计特征。

(4) 另外，还加多一个equivalent sound level。

(5) 所以总共得到14+11+1=26个扩展特征，加上原GeMAPS的62个特征，得到88个特征，这88个特征就是eGeMAPS的特征集。

当前使用的是 eGeMAPSv02，是当前最优的音频提取特征集

With v2.0.0

Name	#features
ComParE_2016	65 / 65 / 6373
GeMAPSv01a	18 / - / 62
GeMAPSv01b	18 / - / 62
eGeMAPSv01a	23 / - / 88
eGeMAPSv01b	23 / - / 88
eGeMAPSv02	25 / - / 88

归一化

Min-Max Scaling

- **定义**：将所有特征缩放到[0,1]区间内

相比于standard方法，该归一化方法保留了原始数据的分布形状，可以处理所有特征的极端值，并且简单易实现。

针对此音频情感分析，所有的数据都在同一量表上，并且不含特别多的异常值或极端值，特别在后续的决策树上就可以实现很好的拟合，因此使用minmax归一化方法更好。

模型训练

本实验旨在通过音频单模态数据实现情感识别，在模型训练上，尝试使用了不同的分类模型和不同的参数，使用了多种分类模型，并且使用**软硬投票**或者其他集成方法进行集成，并且使用贝叶斯优化进行参数调整，下面进行详细地介绍我已经做过的工作。

1. 决策树分类器

决策树通过学习决策规则对数据进行分类。作为一种直观的非参数化模型，决策树易于解释但易过拟合。本实验中，决策树的深度和分裂标准等超参数通过交叉验证进行优化。

决策树通过递归地选择最优特征进行分裂，构建树状结构的模型，其分裂标准使用 **信息增益** 和 **基尼不纯度** 评估分裂效果。

决策树自身过拟合训练模型，得到的模型虽然不是最好，但是性能相对其他没有调参的模型来说比较好。

2. 随机森林分类器

随机森林 是一种集成学习技术，通过构建多个决策树并集成它们的预测结果以提高分类性能。这种方法能够提供特征重要性的估计，并在一定程度上减少过拟合的风险。在本实验中，使用随机森林对训练数据进行拟合，并利用交叉验证方法优化树的数量和深度等超参数。

随机森林通过 **自助聚合** (bagging) 提高决策树的稳定性和准确性，每棵树在训练时从原始数据集中随机抽取样本，增加了模型的泛化能力。

通过随机森林，将多个弱的决策树分类器，聚合成了一个有效的强分类器，性能也确实有了较大百分点的提高。

3. 逻辑回归

逻辑回归是一种线性模型，用于二分类问题。它通过最大似然估计来确定模型参数。尽管模型假设线性关系，但通过正则化技术如L1和L2，逻辑回归可以适应更复杂的数据结构，在实验探讨了正则化策略对模型性能的影响。

逻辑回归模型通过Sigmoid函数将线性回归的输出映射到(0,1)区间，表示概率预测，正则化技术用于控制模型复杂度，防止过拟合。

该方法是最简单的方法，针对这个数据集，效果并不是很好，可能是其只能线性表达、无法模拟较高维度的特征，导致用其在数据集上表现不好。

4. 朴素贝叶斯分类器

朴素贝叶斯是基于贝叶斯定理的分类方法，假设特征条件独立。该方法在处理大量类别特征时尤为有效，尽管其独立性假设可能不成立。在本实验中，朴素贝叶斯分类器被用于评估其在音频情感识别任务中的效能。

朴素贝叶斯分类器基于概率论，通过计算后验概率进行分类，其“朴素”假设简化了特征间的联合概率计算，降低了计算复杂度。

该方法如果能够保证各个特征尽可能地独立，则会有较好性能的分类效果，甚至可以达到90%以上，但是现实生活中这种特征集难以获取。

5. AdaBoost分类器

AdaBoost是一种提高弱分类器性能的集成方法，通过迭代关注误分类样本来增强模型性能。本实验中，AdaBoost被应用于构建一个强分类器，并考察了不同弱学习器对最终性能的影响。

AdaBoost通过调整数据权重，使模型在训练过程中更加关注之前被误分类的样本，从而提高整体分类性能。

该方法也相当于使用集成方法的内容，通过迭代关注误分类样本来增强模型性能，将不同的弱学习器集成为效果较好的强分类器。

6. 支持向量机

支持向量机是一种监督学习技术，旨在高维空间中寻找最优的决策边界，以最大化两个类别间的间隔。SVM特别适用于小样本和非线性问题，通过核技巧可扩展至非线性分类。本实验中，SVM模型的参数通过贝叶斯优化进行调整，以寻找最佳的正则化参数C、核函数类型及其相关参数。

SVM基于最大化间隔原则，通过引入拉格朗日乘子和核函数，将数据线性不可分问题转化为高维空间中的线性可分问题。

这个方法是效果最好的，针对小规模数据集，可以很好地在高维空间中寻找最优的决策边界，从而最大化两个类别间的间隔。

7. 高斯过程分类器

高斯过程分类器是一种基于概率的非参数贝叶斯方法，通过在整个函数空间上定义高斯过程来模拟数据的不确定性。本实验中，高斯过程分类器被应用于音频情感识别任务，并评估了其性能。

高斯过程分类器通过定义先验分布和似然函数，利用贝叶斯定理进行后验推断，适用于小样本问题和不确定性建模。

在实验中，显示该方法的性能并不是很好，该方法主要利用概率，结合贝叶斯定理进行后验推断。

8. 集成学习方法

集成学习通过结合多个学习器的预测结果来提高整体性能。本实验探索了 `软硬投票` 和 `Stacking` 等集成技术，以降低泛化误差并增强模型鲁棒性。

实验中使用了该方法，但是最后没有将多个性能优异的模型进行再次整合，我认为再次整合很可能会继续提高模型的性能。

9. CatBoost分类器

CatBoost是一种梯度提升框架，能有效处理类别特征，并自动进行特征转换。本实验中，CatBoost应用于音频情感识别任务，以评估其在处理复杂数据结构中的潜力。

CatBoost通过优化目标函数的梯度，自动处理类别特征，减轻模型对数据预处理的依赖，增强了模型的泛化能力。

10. auto-sklearn

`auto-sklearn` 集成了多种机器学习算法，包括传统的机器学习算法（如随机森林、支持向量机等），不仅提供模型训练，还提供了模型评估功能，它可以自动进行交叉验证，并报告多种性能指标，如准确率、F1分数等，帮助评估模型的性能。

在解决过程中，尝试使用这种方法进行自动化调整参数和尝试模型。

在模型训练阶段，我们首先对数据执行预处理和特征提取，随后采用上述算法进行训练和验证。通过交叉验证和网格搜索技术，优化了各模型的超参数，特别是使用了贝叶斯优化来加快超参数的搜索，以实现最佳性能。最终，基于测试集上的性能评估，选择了表现最优的模型，这种方法大概的性能在57%左右

性能评估

使用 accuracy、recall、f1、precision、Confuse_matrix 来评估模型的性能，并且以 f1 为最终评价模型好坏的重要因素。

	决策树	随机森林	二次判别分析	朴素贝叶斯	AdaBoost	逻辑回归	SVM
accuracy	0.4239	0.5344	0.5043	0.4607	0.4859	0.4413	0.5965
recall	0.4228	0.5383	0.5135	0.4945	0.4961	0.4362	0.5961
f1	0.4235	0.5435	0.5122	0.4711	0.4877	0.4304	0.5972
precision	0.4259	0.5619	0.5557	0.4771	0.4944	0.5233	0.6198

(除去集成方法，其他方法使用了贝叶斯优化进行参数优化，四舍五入保留小数点四位)

此外，还使用了集成学习（使用随机森林与 XGBoost）、高斯过程分类器、catBoost，但是效果并不理想，因此在此不进行展示，

auto-sklearn方法得到的集成结果也大概在0.57左右。

通过在测试集上评估，SVM分类器结合贝叶斯优化方法达到了0.5965的准确率，0.5961的召回率，0.5972的F1分数，以及 0.6198的精确率。与其他传统机器学习方法相比，SVM在各项指标上均表现最佳,达到了我测试的sota,但是明显地，使用集成学习将多个有效的弱分类器整合将会得到更强性能的强分类器，可以在后续继续进行尝试。

三.实验数据与设置

实验框架

- 限定使用框架：
 - 机器学习scikit-learn
 - 深度学习pytorch
- 推荐使用版本：
 - python 3.8
 - torch 2.0.1
 - torchaudio 2.0.2
 - torchvision 0.15.2

数据集说明

总数量：5531

train	dev	test
3259	1031	1241

CSV文件说明

使用“#”分隔

num	id	path	text	label
文件数量序号（根据部分对话先后顺序排列）	文件名	音频文件相对路径	音频的文本内容	{0, 1, 2, 3}

label	含义
0	angry
1	happy or excited
2	neutral
3	sad

四.实验结果与分析

实验中，我们首先使用 `opensmile` 库提取音频特征，然后利用 `scikit-learn` 中的随机森林分类器进行模型训练。通过交叉验证和网格搜索对模型参数进行调优，最终在测试集上评估模型性能。实验结果显示，模型在测试集上达到了较高的准确率和F1分数，表明所提方法能够有效识别不同情感状态。

最终选择了SVM+贝叶斯优化方法

该方法在调参时使用了贝叶斯优化，针对 `c`, `gamma`, `kernel`, `degree`, `coef0`, `class_weight`, `tol` 等参数进行最优参数搜索，调试过程中发现，学习率选择`c=0.01`比较好，正则化强度适中，`gamma`选择`scale`比较好，`kernel`核选择`poly`更好，可能是能提高多项式核的度数来增强模拟在高维度上的拟合效果。`degree`大概选择6比较好，这是一种适中的值，但是也能根据数据集进行简单分析选择该值的原因，区分了四种情绪和男女的模型更好，但是模型上可能有重叠，所以接近4-8的度数比较好。`class_weight`会选择`None`，因为该模型上各类别是不平衡的。


```
# def __init__(self, kernel='rbf', C=1.0, gamma='scale'):
#     self.model = SVC(kernel=kernel, C=C, gamma=gamma)
#     self.param_space = {
#         'C': Real(0.01, 2.00, prior='uniform'), # C的范围
#         'gamma': Categorical(['scale', 'auto']), # gamma的选择
#         'kernel': Categorical(['rbf', 'linear', 'poly', 'sigmoid']), # 核函数类型
#         'degree': Integer(2, 8), # 仅对poly核有效, 表示多项式核的度数
#         'coef0': Real(-1.0, 1.0), # poly和sigmoid核的偏置项
#         'class_weight': Categorical([None, 'balanced']), # 处理类别不平衡
#         'tol': Real(1e-5, 1e-1, prior='log-uniform'), # 容忍度范围, 采用对数均匀分布
#     }
```

```
import opensmile
import pandas as pd
import os
import sklearn
import matplotlib.pyplot as plt
import numpy as np
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score, confusion_matrix

## 提取特征函数
## file_list: 音频文件路径的列表 list类型
## 返回值numpy.ndarray 形状:(len(file_list),88)
def extract_audio_feature(file_list, save_path):
    # 如果已存在保存的特征文件, 则直接加载
    if os.path.exists(save_path):
        print(f"已找到保存的特征文件 '{save_path}', 正在加载...")
        feature = np.load(save_path)
        print("特征加载完毕!")
        return feature
    print("请耐心等待特征提取完!")
    smile = opensmile.Smile(
        feature_set=opensmile.FeatureSet.eGeMAPsv02,
        feature_level=opensmile.FeatureLevel.Functionals)#指定了特征集(eGeMAPsv02)和特征级别(Functionals)
    feature = []
    for n, file in enumerate(file_list):#enumerate函数用于遍历, 返回一个迭代器(索引和值)
        y = smile.process_file(file)#调用process_file函数处理音频文件
        y = y.to_numpy().reshape(-1)#-1 表示自动计算这一维的大小
        feature.append(y)
        if (n+1)%100 == 0:
            print(f"当前进度{n+1}/{len(file_list)}")
    print("此次特征提取已结束")
    print("-----")
    feature = np.stack(feature, axis = 0)
    # 将特征保存到指定的文件中
    np.save(save_path, feature)
    print(f"特征已保存到文件 '{save_path}'")
```

```
return feature
```

```
def normalize_features(features, scaler=None, method='minmax'):
```

```
    """
```

```
    对已提取的特征进行归一化或标准化处理，并应用训练集的归一化参数。
```

```
    参数:
```

```
    features: numpy.ndarray, 待归一化的特征数据，形状为 (n_samples, n_features)
```

```
    scaler: 已拟合的归一化器（标准化或归一化器），如果没有提供，则会根据训练集拟合一个新的
```

```
    method: str, 归一化方法, 'standard'为标准化（Z-score），'minmax'为最小-最大归一化，默认 'standard'
```

```
    返回:
```

```
    numpy.ndarray: 归一化后的特征数据
```

```
    """
```

```
    if method == 'standard': # 使用标准化 (Z-score)
```

```
        if scaler is None: # 如果没有提供 scaler, 就拟合一个新的
```

```
            scaler = StandardScaler()
```

```
            features = scaler.fit_transform(features)
```

```
        else: # 如果提供了 scaler, 就应用已经拟合好的归一化参数
```

```
            features = scaler.transform(features)
```

```
        print("标准化 (Z-score) 完成")
```

```
    elif method == 'minmax': # 使用最小-最大归一化
```

```
        if scaler is None:
```

```
            scaler = MinMaxScaler()
```

```
            features = scaler.fit_transform(features)
```

```
        else:
```

```
            features = scaler.transform(features)
```

```
        print("最小-最大归一化完成")
```

```
    else:
```

```
        raise ValueError("未知的归一化方法，请选择 'standard' 或 'minmax'。")
```

```
    return features, scaler
```

```
def calculate_score_classification(preds, labels, average_f1='macro'): #
```

```
weighted, macro
```

```
    accuracy = accuracy_score(labels, preds)
```

```
    f1 = f1_score(labels, preds, average=average_f1, zero_division=0)
```

```
    precision = precision_score(labels, preds, average='macro', zero_division=0)
```

```
    ua = recall_score(labels, preds, average='macro', zero_division=0)
```

```
    confuse_matrix = confusion_matrix(labels, preds)
```

```
    return accuracy, ua, f1, precision, confuse_matrix
```

```
# 贝叶斯优化
```

```
# from sklearn.svm import SVC
```

```
# from sklearn.model_selection import train_test_split
```

```
# from skopt import BayesSearchCV # 导入贝叶斯优化类
```

```
# from skopt.space import Real, Categorical, Integer
```

```
# from skopt.utils import use_named_args
```

```
# from skopt import BayesSearchCV
```

```
# from skopt.space import Real, Categorical, Integer
```

```

# from sklearn.svm import SVC

# class MyRF:
#     def __init__(self, kernel='rbf', C=1.0, gamma='scale'):
#         self.model = SVC(kernel=kernel, C=C, gamma=gamma)
#         self.param_space = {
#             'C': Real(0.01, 2.00, prior='uniform'), # C的范围
#             'gamma': Categorical(['scale', 'auto']), # gamma的选择
#             'kernel': Categorical(['rbf', 'linear', 'poly', 'sigmoid']), # 核函数类型
#             'degree': Integer(2, 6), # 仅对poly核有效, 表示多项式核的度数
#             'coef0': Real(-1.0, 1.0), # poly和sigmoid核的偏置项
#             'class_weight': Categorical([None, 'balanced']), # 处理类别不平衡
#             'tol': Real(1e-5, 1e-1, prior='log-uniform'), # 容忍度范围, 采用对数均匀分布
#         }

#     def train(self, features, labels):
#         print("开始贝叶斯优化...")

#         # 使用贝叶斯优化来寻找最佳参数
#         bayes_search = BayesSearchCV(self.model, self.param_space, n_iter=50,
#                                       cv=5, scoring='accuracy', verbose=1)
#         bayes_search.fit(features, labels)

#         print("贝叶斯优化完成, 最佳参数: ", bayes_search.best_params_)
#         self.model = bayes_search.best_estimator_
#         print("SVM模型训练完成!")

#     def evaluate(self, features, labels):
#         # 在验证集上评估模型
#         print("开始评估模型...")
#         preds = self.model.predict(features)
#         accuracy, recall, f1, precision, conf_matrix =
# calculate_score_classification(preds, labels)
#         return accuracy, recall, f1, precision, conf_matrix

from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from skopt import BayesSearchCV # 导入贝叶斯优化类
from skopt.space import Real, Categorical, Integer
from skopt.utils import use_named_args

from skopt import BayesSearchCV
from skopt.space import Real, Categorical, Integer
from sklearn.svm import SVC

class MyRF:
    def __init__(self):
        # 直接使用您提供的最佳参数
        self.model = SVC(
            C=0.01,
            class_weight=None,
            coef0=0.5349030209235746,
            degree=6,

```

```

        gamma='scale',
        kernel='poly',
        tol=0.0006473353953304592
    )

    def train(self, features, labels):
        print("开始训练SVM模型...")
        self.model.fit(features, labels)
        print("SVM模型训练完成!")

    def evaluate(self, features, labels):
        print("开始评估模型...")
        preds = self.model.predict(features)
        accuracy, recall, f1, precision, conf_matrix =
calculate_score_classification(preds, labels)
        return accuracy, recall, f1, precision, conf_matrix

## 读取train.csv、dev.csv
train_csv = pd.read_csv("./CSVfile/train.csv", sep = "#")
dev_csv = pd.read_csv("./CSVfile/dev.csv", sep = "#")
## 分离文件路径和标签
train_path = list(train_csv.path)
train_label = list(train_csv.label)
dev_path = list(dev_csv.path)
dev_label = list(dev_csv.label)

## test_preds 长度为1241的list, 对应测试集中1241个样本的标签
##运行后会在当前目录生成result.csv文件, 提交result.csv文件即可
##如果没有生成, 请检查test_preds的长度是否为1241!
def write_result(test_preds):
    if len(test_preds) != 1241:
        print("错误! 请检查test_preds长度是否为1241!!!")
        return -1
    test_csv = pd.read_csv("./CSVfile/test.csv", sep="#")
    test_csv["label"] = test_preds
    test_csv.to_csv("./result.csv", sep = "#")
    print("测试集预测结果已成功写入到文件中!")

## 特征处理函数, 可以对提取的特征进行处理, 以获得更好的特征表示
def feature_process(feature):
    return feature

## 主函数
if __name__ == "__main__":
    ## 实例化模型
    rf = MyRF()
    ## 提取训练样本特征
    ## 文件数量很多时需要的时间较长, 请耐心等待
    train_save_path="./feature/train_feature.npy"
    train_feature = extract_audio_feature(train_path, train_save_path) ## np.array
(n,88)
    train_feature_normalized, scaler = normalize_features(train_feature,
method='minmax')
    train_feature = feature_process(train_feature_normalized)

    ##训练模型

```

```

rf.train(train_feature_normalized,train_label)
##评估在训练集上的拟合效果
acc,ua,f1,pre,confuse_matrix =
rf.evaluate(train_feature_normalized,np.array(train_label))
print(f"train:\nAcc:{acc} \nUa:{ua} \nMacro_F1:{f1} \nPre:
{pre}\nConfuse_matrix:\n{confuse_matrix}")

##计算在dev上的性能
dev_save_path="./feature/dev_feature.npy"
dev_feature = extract_audio_feature(dev_path,dev_save_path)
dev_feature_normalized, _ = normalize_features(dev_feature, scaler=scaler,
method='minmax')
dev_feature = feature_process(dev_feature_normalized)
acc,ua,f1,pre,confuse_matrix =
rf.evaluate(dev_feature_normalized,np.array(dev_label))
print(f"dev:\nAcc:{acc} \nUa:{ua} \nMacro_F1:{f1} \nPre:
{pre}\nConfuse_matrix:\n{confuse_matrix}")

#rf.show_and_save_models("model_info.txt")
##读入test.csv
test_csv = pd.read_csv("./CSVfile/test.csv",sep = "#")
test_path = list(test_csv.path)
test_save_path="./feature/test_feature.npy"
test_feature = extract_audio_feature(test_path,test_save_path)
test_feature_normalized, _ = normalize_features(test_feature, scaler=scaler,
method='minmax')
test_label = rf.model.predict(test_feature_normalized)
test_preds = test_label
print(len(test_feature))

## 通过模型获得测试集对应的标签列表test_preds，并写入到result.csv文件中
write_result(test_preds)

```

方法优势分析

1. **特征提取的有效性：** eGeMAPSv02 特征集是情感识别领域中一种先进的音频特征提取方法，它基于其前身 GeMAPS 特征集进行了扩展，包含了广泛的音频特征。这些特征不仅包括传统的频谱特征、基频特征和能量特征，还涵盖了更多的统计特征和时间特征，从而提供了一个全面的音频信号描述。这种特征集的有效性在于其能够捕捉到语音信号中细微的情感变化，这些变化往往是情感状态的直接体现。在音频情感识别中，频谱特征如 梅尔频率倒谱系数 能够描述声音的频谱包络，而基频特征如 音高 则与说话人的语调直接相关。能量特征反映了语音信号的强度变化，而统计特征则能够描述语音信号的变异性 and 复杂性。 eGeMAPSv02 特征集通过综合这些特征，提供了一个更为丰富和细致的音频信号描述，从而提高了情感识别的准确性。
2. **SVM分类器的优势：** 支持向量机（SVM）是一种基于间隔最大化原则的监督学习方法，它通过在高维空间中寻找最优的决策边界来区分不同的类别。其核心优势在于其优秀的泛化能力和对非线性问题的高效处理。通过使用不同的核函数，SVM能够将原始的非线性问题映射到高维空间中，使其线性可分，从而找到最佳的分类边界。SVM中的核技巧类似于深度学习中的卷积操作，它能够捕捉局部特征并将其映射到高维空间中，以便于区分不同的类别。类似于在图像处理中，高斯核对局部区域的特征进行编码，类似于卷积神经网络中的滤波器，能够提取图像中的关键信息。在音频情感识别中，SVM通过核技巧能够有效地处理复杂的情感边界问题，提高了分类的准确性。

3. **贝叶斯优化的参数调整**：贝叶斯优化是一种基于概率模型的全局优化方法，它通过构建目标函数的概率模型来指导搜索最优参数。这种方法的核心在于它能够利用已有的评估信息来预测未知区域的函数值，从而以更少的迭代次数找到最优解。相比于传统的网格搜索或随机搜索，贝叶斯优化更加高效，因为它能够根据已有的评估结果动态调整搜索策略，避免了在非优区域的无效搜索。贝叶斯优化的另一个优势在于它能够处理参数空间的不确定性，通过构建概率模型来描述参数空间的分布，从而在不确定性中寻找最优解。这种方法在机器学习模型的参数调优中尤为重要，因为它能够显著提高模型训练的效率和效果。
4. **数据预处理的重要性**：归一化处理是机器学习中的一个重要步骤，它能够提高模型训练的效率和效果。在音频情感识别中，由于不同特征的量表可能差异很大，归一化处理能够确保所有特征在模型训练中具有相同的重要性，避免了某些特征由于尺度较大而在模型训练中占据主导地位。在本实验中，采用了Min-Max Scaling进行归一化处理，这种处理方式保留了原始数据的分布形状，并且对异常值不敏感。这使得模型能够更好地处理不同量表上的特征，提高了模型的泛化能力。

模型解释性

1. 数据集特性对模型解释性的影响

- **特征分布**：数据集中特征的分布情况对SVM模型的解释性有直接影响。特征分布的不均匀性或极端值可能导致模型在某些区域过于敏感，从而依赖特定的支持向量进行分类。这种依赖性会降低模型的泛化能力，因为模型可能过度适应训练数据中的特定模式，而不是学习到更广泛的、可推广的模式。在SVM中，这意味着模型可能在某些特征方向上具有更高的敏感度，导致决策边界在这些方向上更加复杂。
- **类别不平衡**：数据集中的类别不平衡问题可能导致模型偏向于多数类，从而影响模型的解释性。特别是在情感识别等任务中，某些类别的样本可能远多于其他类别。这种不平衡可能导致模型偏向于多数类，从而忽视少数类的特征和模式。在SVM中，通过调整 `class_weight` 参数，可以给予少数类更高的权重，使模型更加关注这些类别，提高对少数类的识别能力，进而增强模型的公平性和解释性。
- **特征相关性**：数据集中特征之间的相关性也会影响SVM模型的解释性。在SVM中，每个特征都被视为独立的，但实际上，高度相关的特征可能导致模型过度依赖某些特征，忽视其他同样重要的特征。这种依赖性不仅影响模型的性能，也降低了模型的解释性，因为模型的预测可能过于依赖少数几个特征，而不是整个特征集的联合效应。

2. 结合参数和数据集进行模型解释

在实验中，我们使用了 eGeMAPSv02 特征集，这是一个包含88个特征的集合，它综合了多种音频信号特征，能够有效捕捉语音信号中的情感特征。结合SVM模型，我们通过贝叶斯优化选择了最佳的模型参数，如 `C=0.01`、`kernel='poly'`、`degree=6`、`gamma='scale'` 等。这些参数的选择反映了我们对模型性能和解释性的追求：

- **C=0.01 (正则化参数)**：该参数控制误分类点的惩罚程度，较高的 `C` 值会增加对支持向量的惩罚，使得模型更关注于正确分类所有训练样本，可能导致过拟合。较低的 `C` 值则允许更多的误分类点，但可以提高模型的泛化能力。通过分析 `C` 参数的选择，可以实现此模型在偏差和方差之间的权衡，可以通过适当增大正则化参数来减弱模型的过拟合。此参数的选择表明在模型中采取了一种折中的方法，既避免了过拟合，又保持了模型对训练数据的良好拟合，在情感识别任务中，这种适当的正则化可以帮助模型捕捉到更一般化的情感模式，而不是过度适应训练数据中的噪声。
- **Kernel='poly'和Degree=6**：核函数的选择决定了特征空间的映射方式。不同的核函数对模型的解释性有不同的影响，因为它们决定了支持向量在原始特征空间或映射后的特征空间中的分布。多项式核和6次多项式度的选择表明我们假设数据在特征空间中存在非线性关系，这种非线性关系可以通过高次多项式来捕捉，从而允许SVM在高维空间中找到更复杂的决策边界。在情感识别中，使用这种核更有优势，能够模拟高维数据的特征，使得最后模型的性能较好，容易理解，模型区分了四种情绪，如果进一步细分为男女，则最多有8种分类，但是这些之中有重叠，所以超参数优化范围

选择4-8比较合理，最后选择了6，也说明了男女中有情绪是相近的，不需要细分，这种复杂的决策边界有助于区分微妙的情感差异，提高模型的准确性和解释性。

- **Gamma='scale'**: 此参数定义了单个训练样本影响决策边界的范围。较高的 gamma 值会创建一个复杂、曲线更多的决策边界，而较低的 gamma 值则会产生一个平滑的决策边界。gamma 的选择直接影响了模型对训练数据的拟合程度，进而影响模型的解释性。此参数的选择反映了对特征缩放的考虑，'scale' 选项使得 gamma 值与特征的缩放有关，有助于模型更好地处理特征间的差异。在情感识别中，不同特征的情感表达能力可能差异很大，适当的 gamma 设置可以帮助模型平衡这些特征的贡献，提高模型的解释性和鲁棒性。

- **Class_weight=None**

由于数据集中各类别分布不均匀，选择 None 意味着不对类别权重进行调整。这可能会导致模型偏向于多数类，从而影响模型的公平性和解释性，在此数据集下，情绪类别并不是平衡的，因此这样选择是正确的。

五.总结与展望

本实验通过音频单模态和机器学习方法实现了情感识别任务。实验结果表明，所提出的模型在给定数据集上具有良好的性能。但是当前工作仍缺少一些更为细致的考虑：

1. **数据预处理的深化研究**：未来的研究可以探索更为先进的音频预处理技术，针对原音频进行优化，例如根据文件以及音频独立提取出男声和女声，以提高模型的鲁棒性；也可以在提高降噪算法上深入研究，例如基于深度学习的降噪算法，以及针对特定类型的音频数据（如不同性别、年龄和背景噪音）的优化策略，以进一步提升模型的鲁棒性。
2. **特征提取方法的创新**：同时关注于音频特征的提取，尝试更新优化现有的eGeMAPS特征集，也可以探索结合深度学习的特征提取方法，如自编码器或卷积神经网络（CNN），以期获得更具区分力的特征表示，从而提高情感识别的准确性。
3. **模型算法的改进**：针对现有机器学习算法的局限性，未来的研究可以开发新的模型算法，或者结合深度学习技术，提出新的模型架构，以进一步提高情感识别的准确性和鲁棒性。
4. **多模态情感识别的探索**：未来的研究可以考虑融合文本、视频等多种模态信息，进行多模态情感识别研究，这可能为情感状态的全面理解提供更丰富的信息。
5. **模型解释性的研究**：尽管SVM模型具有一定的解释性，但未来的研究可以进一步探索模型解释性的方法，如特征重要性分析、局部可解释模型-agnostic解释（LIME）等，以增强模型的可解释性和透明度。

六.参考资料

[1] [论文笔记：语音情感识别（五）语音特征集之eGeMAPS, ComParE, 09IS, BoAW - PilgrimHui - 博客园](#)

[2] [OpenSmile提取eGeMAPS 特征集-CSDN博客](#)

[3]刘振焄,徐建平,吴敏,曹卫华,陈略峰,丁学文,郝曼,谢桥.语音情感特征提取及其降维方法综述[J].计算机学报,2018,41(12):2833-2851

[4]F. Eyben *et al.*, "The Geneva Minimalistic Acoustic Parameter Set (GeMAPS) for Voice Research and Affective Computing," in *IEEE Transactions on Affective Computing*, vol. 7, no. 2, pp. 190-202, 1 April-June 2016, doi: 10.1109/TAFFC.2015.2457417.

[5]LIU Jiming, ZHANG Peixiang, LIU Ying, ZHANG Weidong, FANG Jie. Summary of Multi-modal Sentiment Analysis Technology[J]. Journal of Frontiers of Computer Science and Technology, 2021, 15(7): 1165-1182.